

Algorithms and Data Structures I.

Practice 5

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

March 14, 2024

Content in this Practice I

- ① Review last week -
Reversed Polish Notation
 - Reversed Polish Notation
 - Dijkstra: Shunting yard algorithm
 - Evaluation from Polish notation
- ② Linked lists - One-way lists
 - Linked lists
 - Simple one-way list (S1L)
 - Iterating and Searching
 - Insertion
 - One-way lists with header node (H1L)
- ③ Illustration problems for One-way lists
 - Reversing an S1L list
 - Sieve of Eratosthenes with S1L

Content in this Practice II

- 4 Linked lists - Two-way (doubly)
 - Simple two-way (doubly) linked list (S2L)
 - Cyclic two-way list (C2L) with header
 - Problem: Sorted list filtered to even and odd sorted

Review last week - Reversed Polish Notation

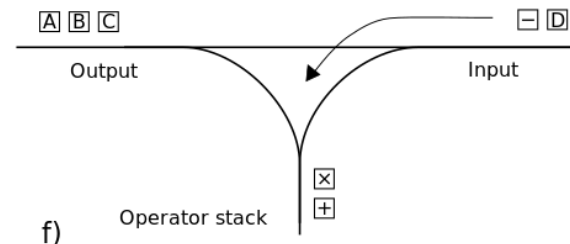
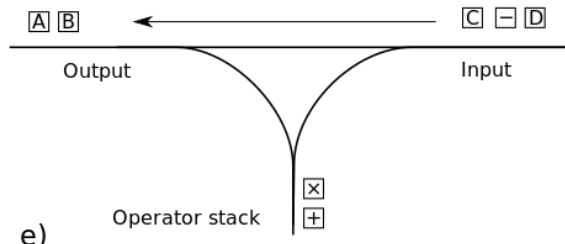
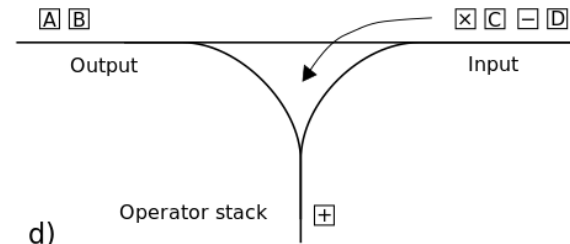
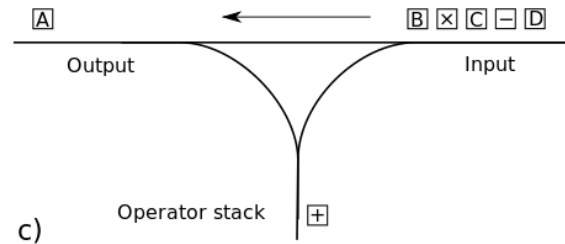
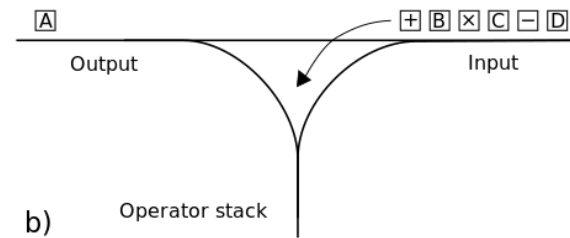
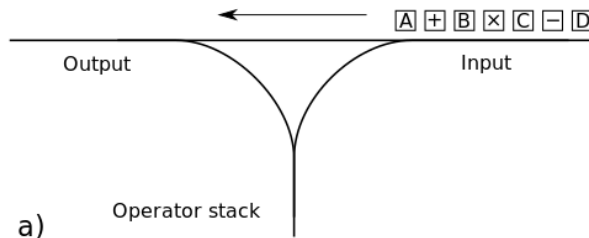
Reversed Polish Notation

- A mathematical notation for evaluating algebraic expressions efficiently (by a compiler).
- The usual '*human*' notation is if, add 3 and 4, it is called **infix** we write
 $3 + 4$
where the **operator** (+) is between the **operands** (3, 4)
- But why don't we write it, where the operators follow their operands, it is called **postfix**
 $3\ 4\ +$
- For multiple operations, instead of
 $3 + 4 * 5 \rightarrow 3\ 4\ 5\ * +$

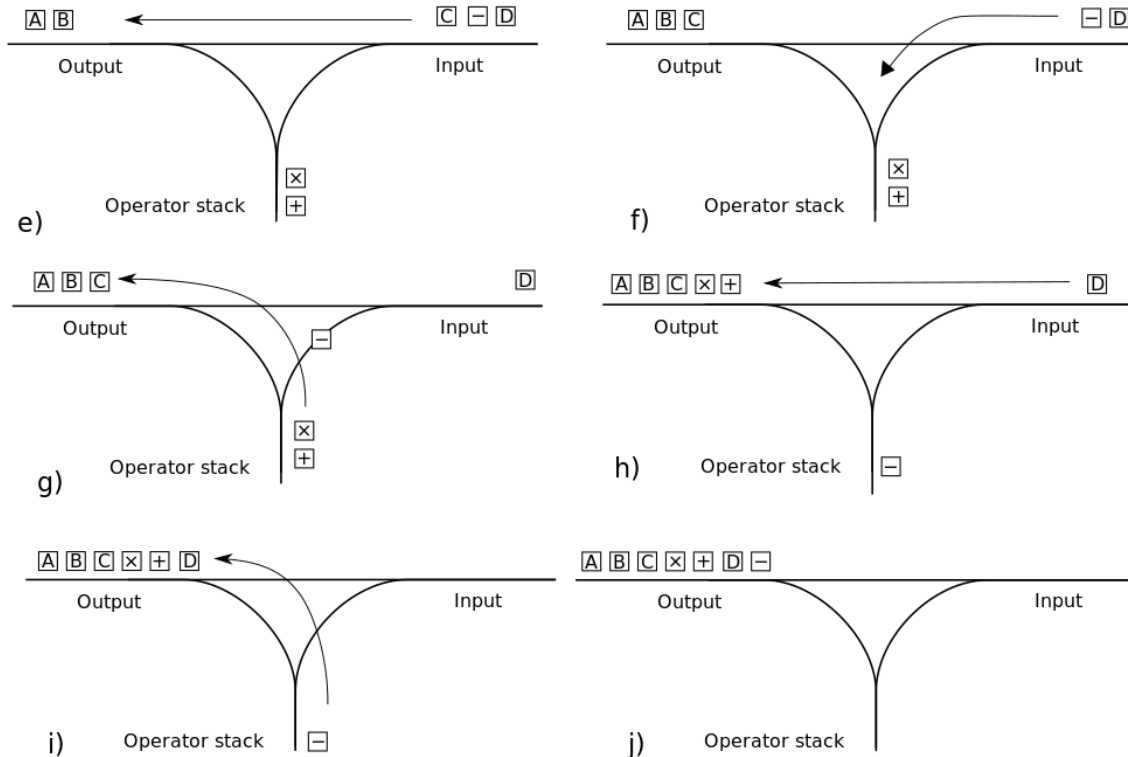
Reversed Polish Notation

- Some examples infix $\rightarrow$ postfix
 - $a+b \rightarrow ab+$
 - $a*b+c \rightarrow ab*c+$
 - $a+b*c \rightarrow abc*+$
 - $a/b*c \rightarrow ab/c*$
 - $(a-b)/(c+d) \rightarrow ab-cd+ /$
 - Single operand operator (neg. sign)

Dijkstra: Shunting yard algorithm



Dijkstra: Shunting yard algorithm



Conversion from infix to postfix

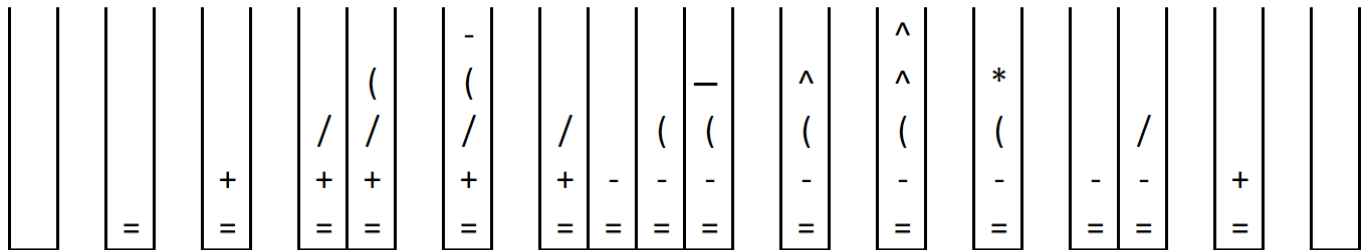
Original expression: $x = b + 2 / (a - c) - (-f \wedge b \wedge 2 * g) / z + w$

Precedence ranking:

$\text{prec}(=) < \text{prec}(+) = \text{prec}(-) < \text{prec}(\wedge) < \text{prec}(/) < \text{prec}(\wedge) < \text{prec}(-)$

Contents of the stack: For each character whatever is currently in the stack

x = b + 2 / (a - c) - (- f ^ b ^ 2 * g) / z + w



Resulting polish notation

x b 2 a c - / + f - b 2 ^ ^ g * z / - w + =

Conversion from infix to postfix

Polish Notation (n:N)				
V : Stack(n)				
read(x)				
$x \in \{ \text{operands} \}$	$x = '('$	$x = ')'$	$x \in \{ \text{LR operators} \}$	$x \in \{ \text{RL operators} \}$
Write(x)	V.push(x)	$V.top() \neq '('$	$\neg V.isEmpty() \wedge V.top() \neq '(' \wedge \text{prec}(V.top()) \geq \text{prec}(x)$	$\neg V.isEmpty() \wedge V.top() \neq '(' \wedge \text{prec}(V.top()) > \text{prec}(x)$
		Write(V.pop())		
		V.pop()	Write(V.pop())	Write(V.pop())
			V.push(x)	V.push(x)
$\neg V.isEmpty()$				
Write(V.pop())				

- LR operators: Set of operators which has precedence from left to right (+ - * / ...)
- RL operators: Set of operators which has precedence from right to left (^ := ...)

Evaluation from Polish notation

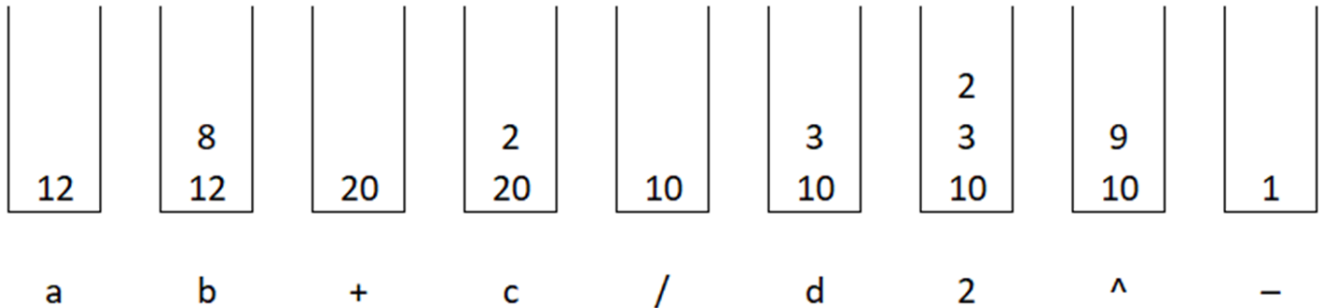
$a \ b \ + \ c \ / \ d \ 2 \ ^ \ -$

$a=12$

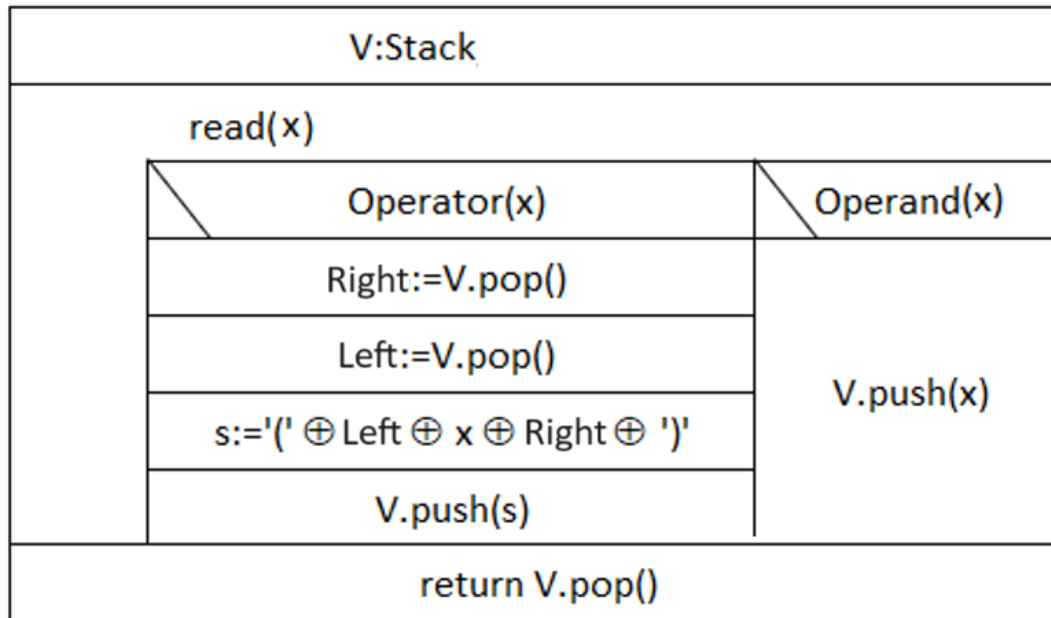
$b=8$

$c=2$

$d=3$



Evaluation from Polish notation



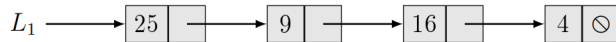
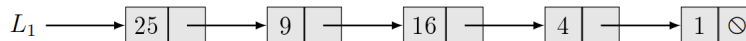
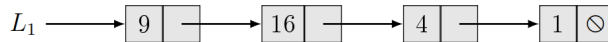
Linked lists - One-way lists

Linked lists

- A **linked list** is a data structure in which the objects are arranged in a linear order. Unlike an array, however, in which the linear order is determined by the array indices, the order in a linked list is determined by a pointer in each object. *Source: Introduction to Algorithms by Cormen et. al.*
- Types:
 - Simple one-way list (S1L)
 - One-way lists with header node (H1L)
 - Cyclic one-way list (C1L)
 - Simple two-way or doubly linked list (S2L)
 - Cyclic two-way list (C2L) with header

Simple one-way list (S1L)

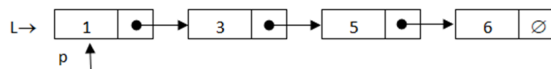
$L_1 = \bigcirc$



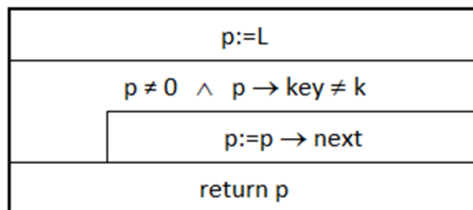
- Each list element has its general data, and a pointer pointing to the next element in the list. If it's the last element, the pointer is NULL.
- L points to the first element, but L is NOT an element in itself, nor is it a type of E1.

E1
$+key : \mathcal{T}$... // satellite data may come here
$+next : E1^*$
$+E1() \{ next := \bigcirc \}$

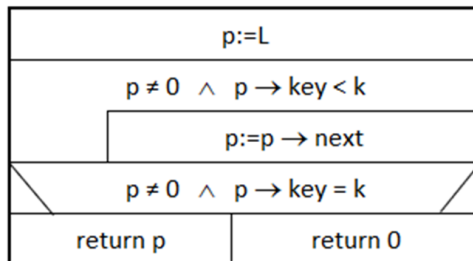
Simple one-way list (S1L) - Iterating and Searching



S1L_search(L:E1*, k:T) : E1*



S1L_search(L:E1*, k:T) : E1*



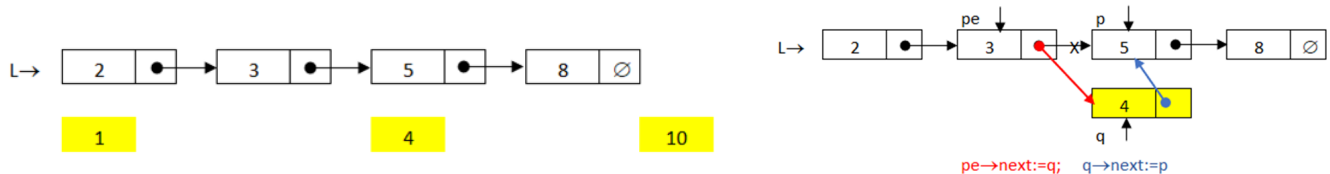
Iterating:

- p: pointer to one of the elements p: E1*
- p→key: the value of the list element.
- p→next: pointer value of the next element (if any).
- Allocating a new element: p:= new E1
- Deleting list element: delete p
 After deletion referencing any fields is invalid.

Searching:

- Let L point to an ordered list of S1L. Find an element in L with key 'k'. If one such element exists, return with its pointer, otherwise return null.

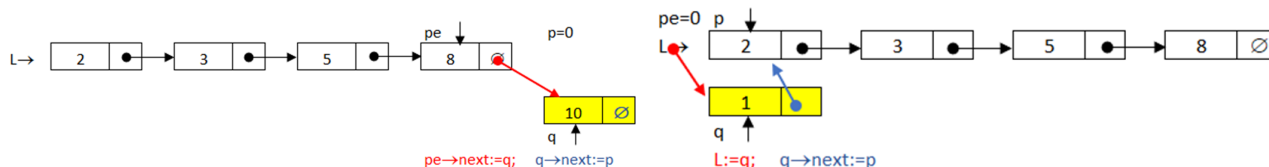
Simple one-way list (S1L) - Insertion



• Insertion:

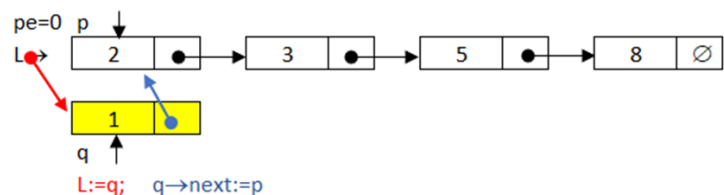
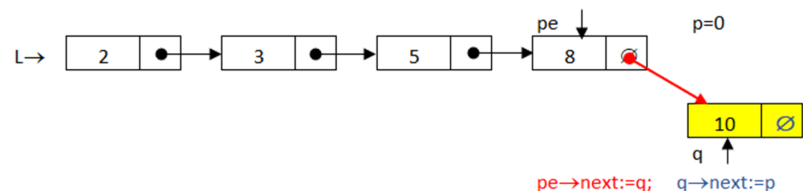
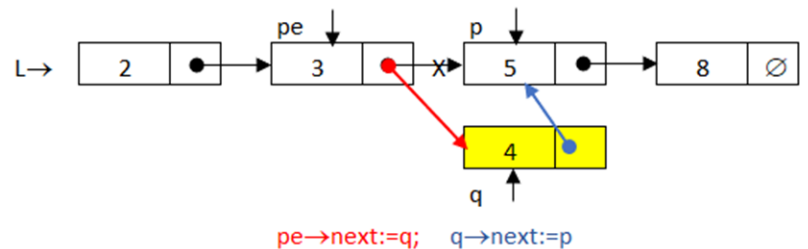
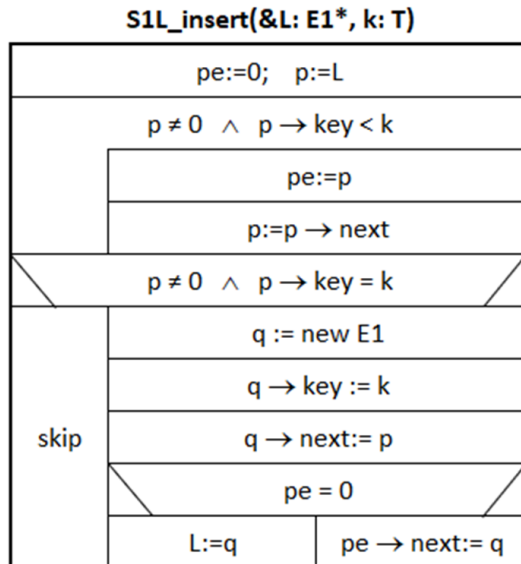
- Let L point to an ordered S1L. Insert a key into the list, and keep it ordered.
- If such key already exists, do nothing. (set)
- The position of the key can be in the four possible places:
- The element should be somewhere in the middle,
- The element is the largest, making it the last,
- The element is the smallest, making it the first,
- The list is empty.

Simple one-way list (S1L) - Insertion

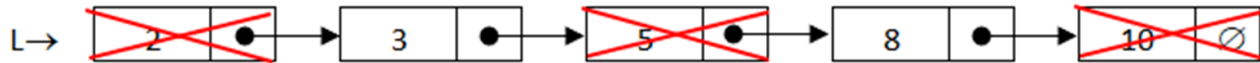


- At the end:
 - Inserting to the end of the list:
 - p is null since every element is smaller than the key to be inserted.
 - pe then is the last element, allowing us to insert after it.
- At the start:
 - Inserting at the beginning means that L should now point to the new element (pe is null):
 - Inserting into an empty list means that both p , and pe are null:

Simple one-way list (S1L) - Insertion



Simple one-way list (S1L) - Deletion

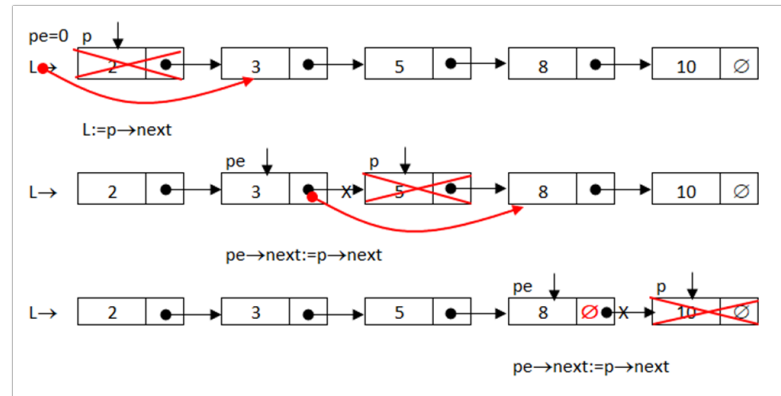
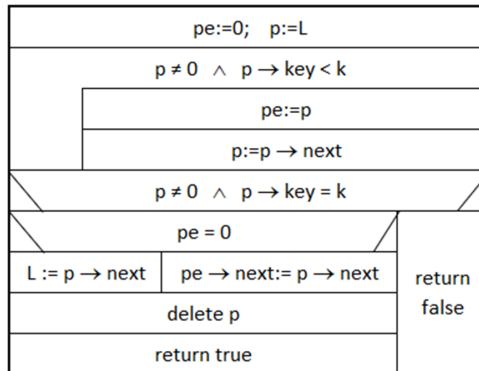


- Deletion:

- Once again, L points to a sorted S1L. Delete an element if it exists.
- If the deletion is successful (exists) return true, otherwise false.
- To delete the element and keep the list intact, we need the previous element once again.
- In case of (successful) deletion, the following cases are possible:
 - Deleting the first element.
 - Deleting an inner element,
 - Deleting the last element.

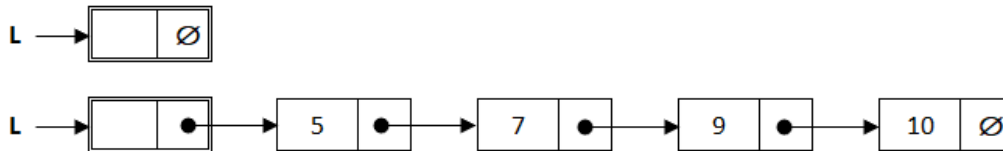
Simple one-way list (S1L) - Deletion

S1L_delete(&L:E1*, k: T):B



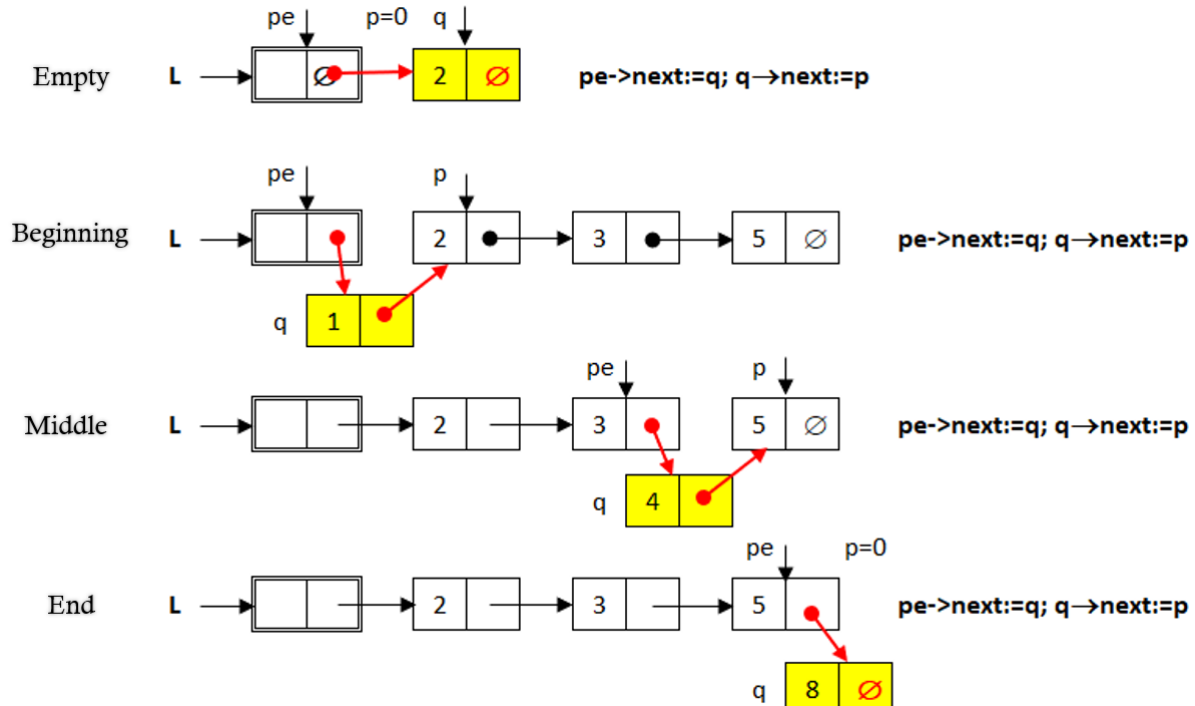
One-way lists with header node (H1L)

- Header element:
 - As we've seen these algorithms need to handle the beginning of the list differently from the other cases since `pe` is null.
 - To solve this issue, and avoid unnecessary branches we introduce a header element.
 - This way the list will always have an `E1` element, sort of a placeholder!
 - This also means that we however don't store data in it.

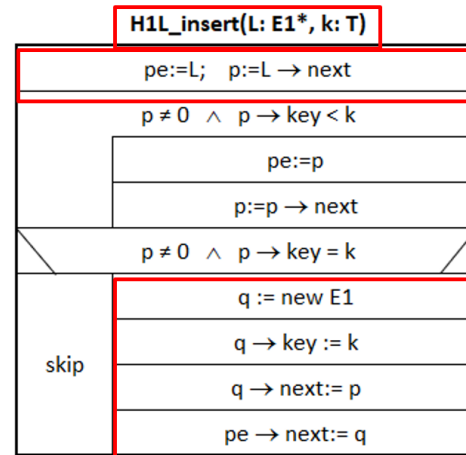
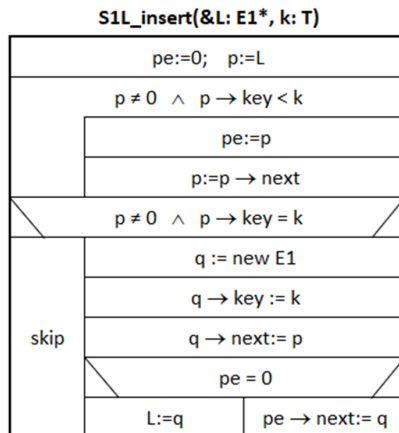


- Searching will be slightly different:
 - We start from the first element after the head

One-way lists with header node (H1L) - Insertion

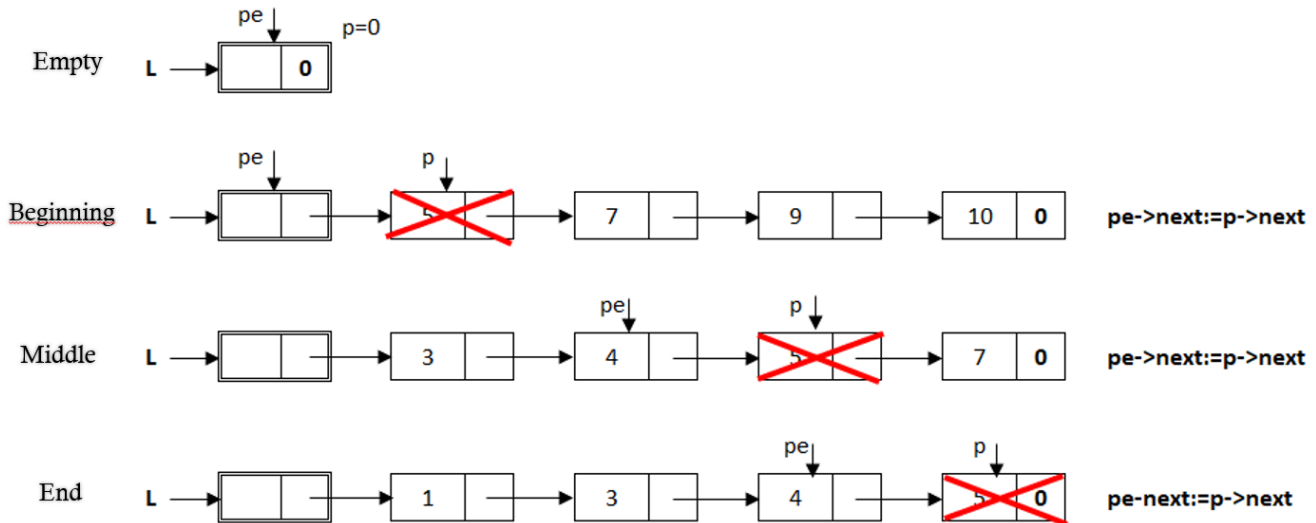


One-way lists with header node (H1L) - Insertion

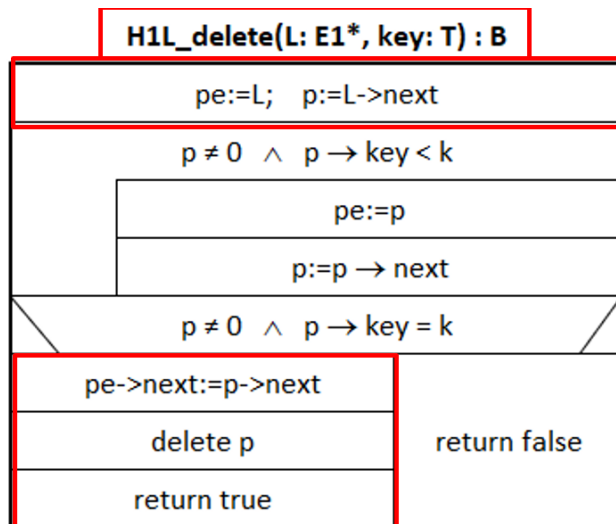
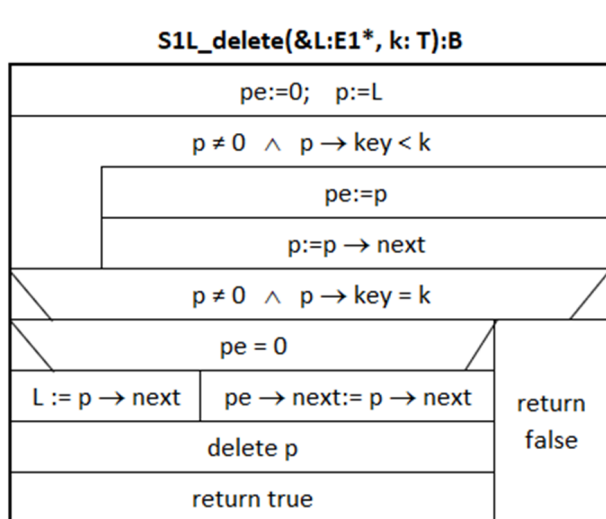


- Differences:
 - L is not passed by reference.
 - Start: pe is the header, p is the first element (can be null)
 - No branch is needed since pe is never null.

One-way lists with header node (H1L) - Deletion



One-way lists with header node (H1L) - Deletion



• Differences:

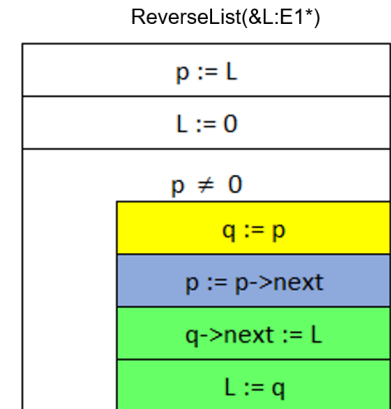
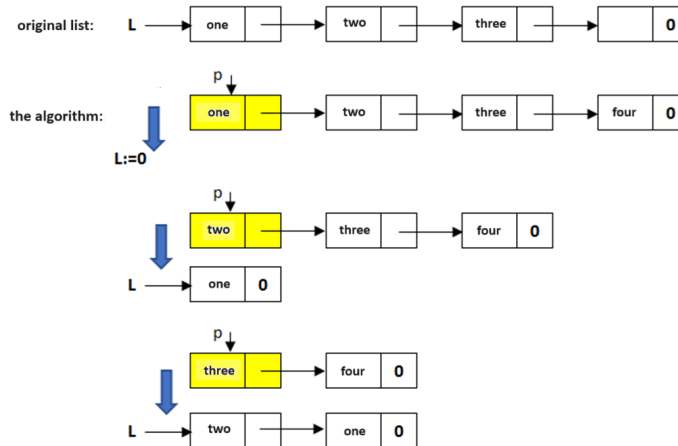
- L is not passed by reference.
- Start: pe is the header, p is the first element (can be null)
- No branch is needed since pe is never null.

Illustration problems for One-way lists

Reversing an S1L list

- Let L point to an S1L. Reverse the order of elements.
- Input:
 - $L \rightarrow \text{one} \rightarrow \text{two} \rightarrow \text{three} \rightarrow \text{four}$
- Output:
 - $L \rightarrow \text{four} \rightarrow \text{three} \rightarrow \text{two} \rightarrow \text{one}$
- Idea:
 - 'Deconstruct' the original list, and p points to the first element.
 - Set L to empty list //L:=0
 - Iterating on the original list, and inserting each element right at the beginning of L will result in the reversed list.

Reversing an S1L list



- q is the element to be inserted at the new beginning
- We must step to the next element in the previous list, otherwise, we'd lose track of the original list
- Inserting q: the previous first element should come after q, and q should be the new first element.

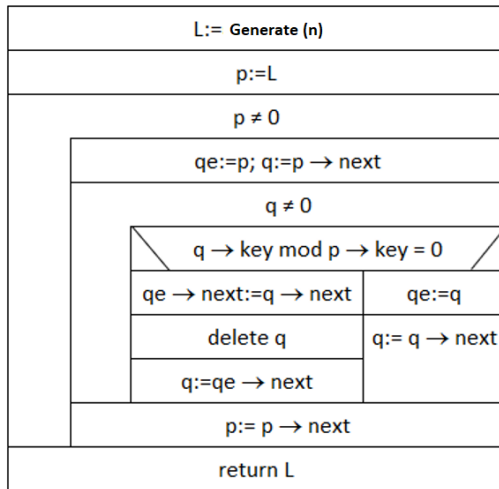
Sieve of Eratosthenes with S1L

- Using Eratosthenes' algorithm create a list of prime numbers smaller than n .
- Idea:
 - Create a list of integers smaller than n . 1 is not a prime, delete. 2 is the first prime, and from now on we have to delete every second element (since they are a multiple of 2, and therefore not primes.)
 - From the remaining elements, the next is 3, it's a prime, keep it, and delete the multiples of 3 once again.
 - And so on until we reach the last element. The remaining elements are all primes.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

Sieve of Eratosthenes with S1L

PrimeSieve(n:N):E1*

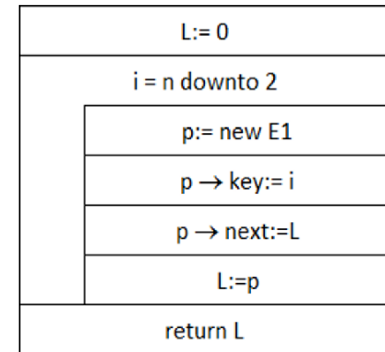


p points to next prime.

q will point to the follow-up elements, and we check if the given element is a multiple of p or not. If yes, delete.

Given that we deleted the multiples of p (and the multiples before it), the next element after p will also going to be a prime.

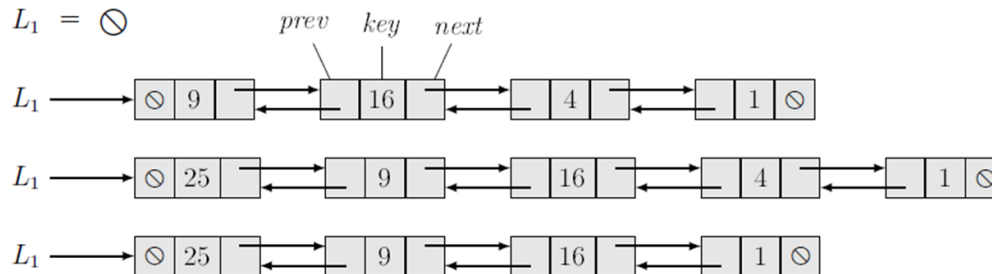
Generate(n:N):E1*



26

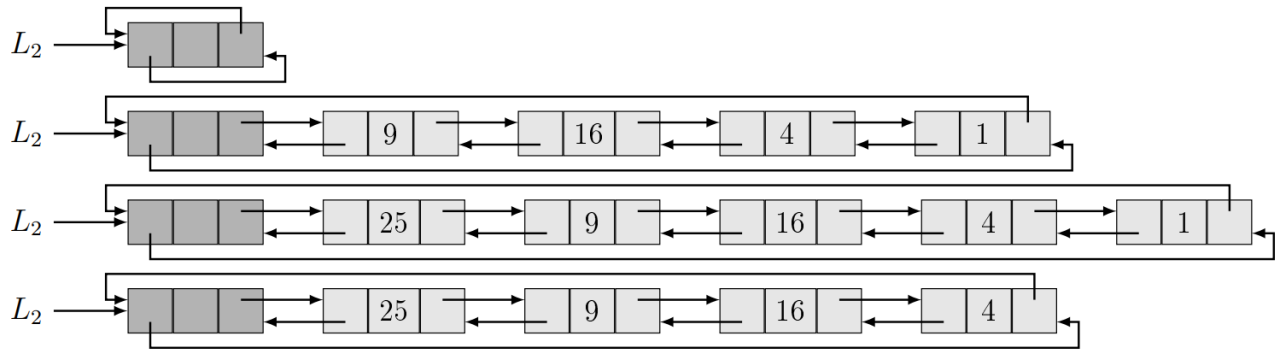
Linked lists - Two-way (doubly)

Simple two-way (doubly) linked list (S2L)



- Advantage: Easy to ‘maneuver’.
- Handling the inner, last, and first elements still, once again requires different handling:
- An element inside the list has *prev* and *next* neighbors, which means we must handle 4 pointers.
- The first element has no previous neighbor, meanwhile the last element has no next.
- Inserting into an empty list means that the element to be inserted won't have any neighbors.

Cyclic two-way list (C2L) with header



- Each element will contain the prev and the next pointer.
- The 'cyclic' aspect refers to the following:
 - The prev value of the head element always points to the last element of the list
 - The next pointer of the last element in the list therefore points to the first element of the list

Cyclic two-way list (C2L) functions, E2 Node type

E2
+ <i>prev, next</i> : E2* // refer to the previous and next neighbour or be this
+ <i>key</i> : $\mathcal{T}$
+ E2() { <i>prev</i> := <i>next</i> := this }

precede(*q*, *r* : E2*)

// (* <i>q</i>) will precede (* <i>r</i>)
<i>p</i> := <i>r</i> → <i>prev</i>
<i>q</i> → <i>prev</i> := <i>p</i> ; <i>q</i> → <i>next</i> := <i>r</i>
<i>p</i> → <i>next</i> := <i>r</i> → <i>prev</i> := <i>q</i>

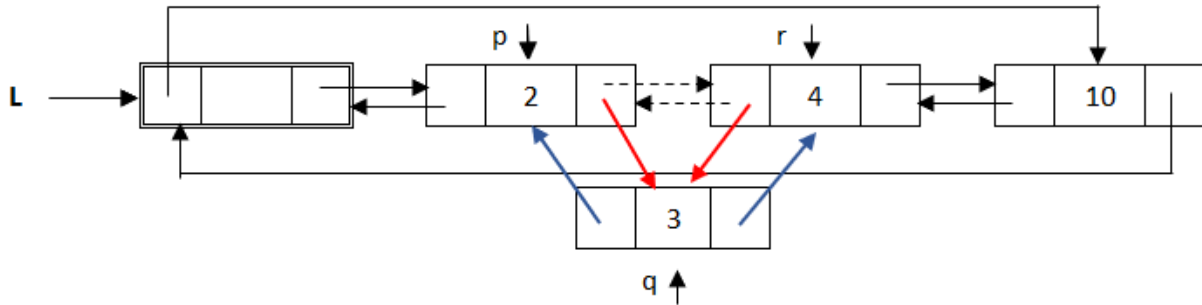
follow(*p*, *q* : E2*)

// (* <i>q</i>) will follow (* <i>p</i>)
<i>r</i> := <i>p</i> → <i>next</i>
<i>q</i> → <i>prev</i> := <i>p</i> ; <i>q</i> → <i>next</i> := <i>r</i>
<i>p</i> → <i>next</i> := <i>r</i> → <i>prev</i> := <i>q</i>

unlink(*q* : E2*)

// remove (* <i>q</i>)
<i>p</i> := <i>q</i> → <i>prev</i> ; <i>r</i> := <i>q</i> → <i>next</i>
<i>p</i> → <i>next</i> := <i>r</i> ; <i>r</i> → <i>prev</i> := <i>p</i>
<i>q</i> → <i>prev</i> := <i>q</i> → <i>next</i> := <i>q</i>

Cyclic two-way list (C2L) functions - precedence



$\text{precede}(q, r : E2^*)$

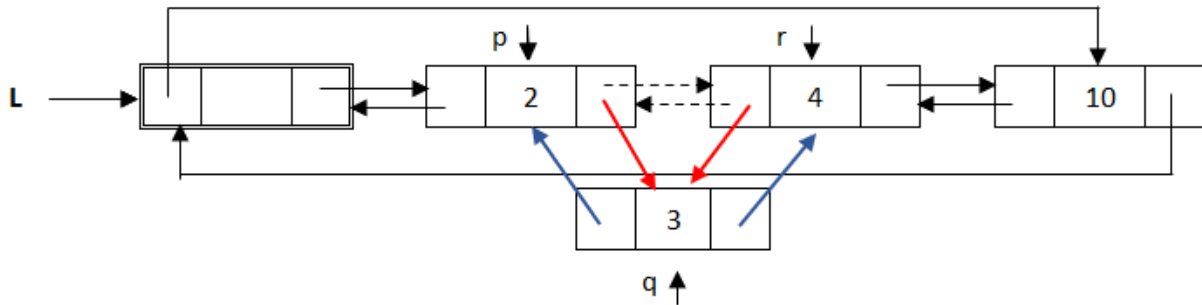
// $(*q)$ will precede $(*r)$

$p := r \rightarrow \text{prev}$

$q \rightarrow \text{prev} := p ; q \rightarrow \text{next} := r$

$p \rightarrow \text{next} := r \rightarrow \text{prev} := q$

Cyclic two-way list (C2L) functions - follow



$\text{follow}(p, q : E2^*)$

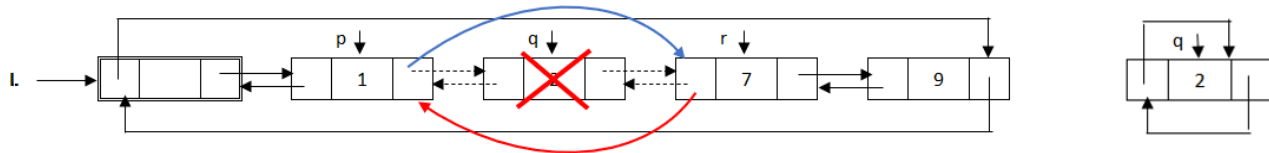
// $(*q)$ will follow $(*p)$

$r := p \rightarrow \text{next}$

$q \rightarrow \text{prev} := p ; q \rightarrow \text{next} := r$

$p \rightarrow \text{next} := r \rightarrow \text{prev} := q$

Cyclic two-way list (C2L) functions - unlink



$\text{unlink}(q : E2^*)$

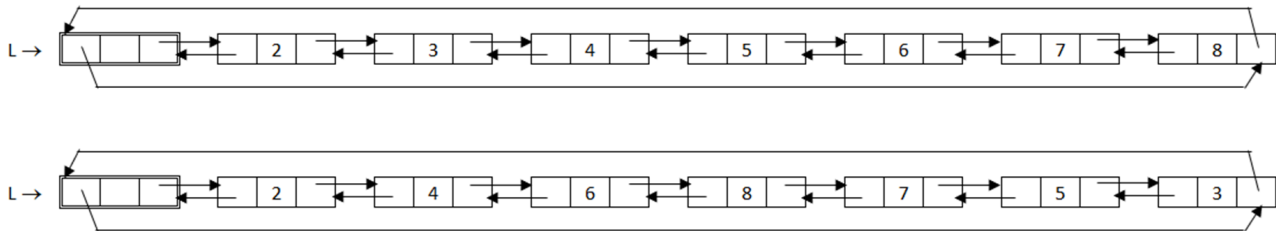
// remove ($*q$)

$p := q \rightarrow \text{prev} ; r := q \rightarrow \text{next}$

$p \rightarrow \text{next} := r ; r \rightarrow \text{prev} := p$

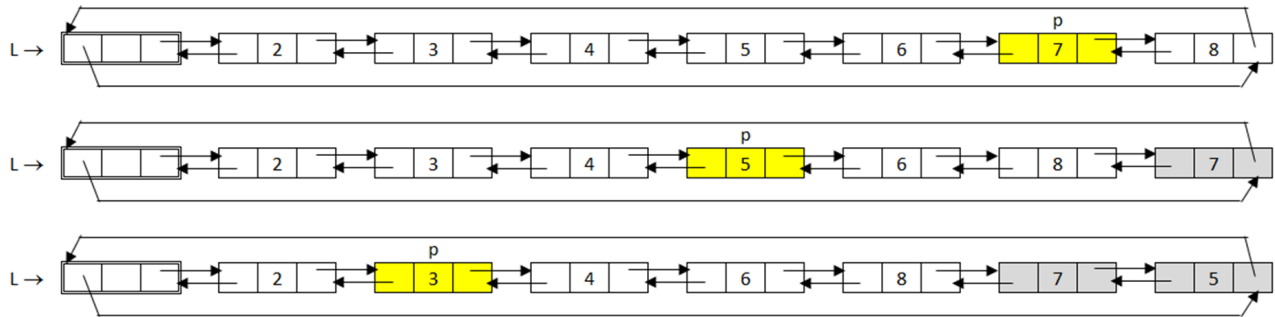
$q \rightarrow \text{prev} := q \rightarrow \text{next} := q$

Sorted list filtered to even and odd sorted



- Let L point to an ordered C2L list containing natural numbers.
- Let's reorder the list so that at the 'front' of the list there are only even numbers, and at the 'end' there are only odd numbers.
- Note: Both 'parts' of the list should remain ordered.

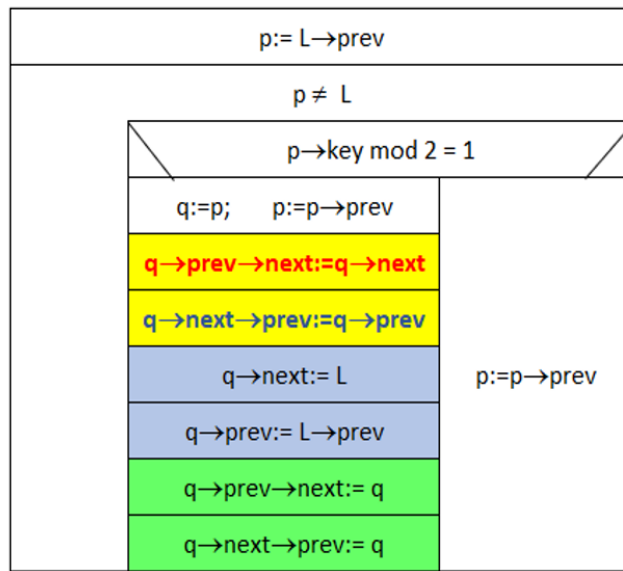
Sorted list filtered to even and odd sorted



- Iterate on the list backward with p.
- If p is odd, we link it before the header (=end of the list)

Sorted list filtered to even and odd sorted

ReorderC2L(L:E2*)



We only have to do anything if p is odd

Yellow: Linking out the odd element

Blue: Inserting q before the header.

Green: Setting the neighbours' pointers

33

Review last week - Reversed Polish Notation
Linked lists - One-way lists
Illustration problems for One-way lists
Linked lists - Two-way (doubly)

Simple two-way (doubly) linked list (S2L)
Cyclic two-way list (C2L) with header
Problem: Sorted list filtered to even and odd sorted

Ending

Thank you for your attention!